CO1 – AT1

Assignment Report

Part 1 of 2 – Tasks 1 to 6

Namma Metro Smart-Card Fare System

Object-oriented analysis, tokens and constants, data types, implicit conversions, operators and control structures

Name	T.R Sangeet
Reg No	1921260245
Course Code	DSA0104
Course Name	OBJECT ORIENTED PROGRAMMING C++

TASK 1 - Understand the Problem and Identify the Objects

4 Marks | K5 Evaluate

1.1 Objects, data and behaviour

The fare problem has a few real-world things that each own some data and some rules. An object is justified only if it owns data that must be kept consistent or behaviour that would otherwise be scattered. The table below lists each candidate and whether it earns a class.

Object	Data (members)	Behaviour (member functions)	Justification / decision
SmartCard	`balance_` (long long, paise, private)	`balance()`, `recharge(p)`, `canTravel(f)`, `deduct(f)`	**Class.** It owns the only value that must never break a rule (balance >= Rs 50 after a journey, never negative). Hiding it makes the rule enforceable in one place.
Passenger	category (Adult / Student / Senior), one card	chooses to travel; supplies category	**Light class / struct.** Only the category affects the fare; no rule needs a private invariant, so it stays a simple aggregate.
Journey	zone, time (minutes since midnight), category	`isPeak()`, `fare()`	**Class / function group.** A journey is an event, not a long-lived thing; it combines zone, time and category into a fare.
Zone	Inner, Middle, Outer	none (enumeration)	**enum class.** A closed set of names with no behaviour; an enum prevents invalid values and mixing with ints.
FareTable	base fares 1000/2000/3000 paise, peak 20 %, concession %	`percentOf(a, p)`; lookup by zone	**Constants in one place (constexpr).** Fare values live here only, so a revision is made once.
Gate	none stored	reads input, calls card + journey, prints result	**Driver code** (`main`). It coordinates objects but owns no rule.

1.2 Procedural style versus object-oriented style

In a procedural program the data are global variables (`balance`, `minBalance`, `fare table`) and the behaviour is free functions (`canTravel(balance, fare)`, `deduct(...)`). The program is a sequence of steps; any function may read or write any variable. In the object-oriented version the balance sits inside `SmartCard` and can be changed **only** through `deduct()` and `recharge()`, which check the rules. The change is therefore from *'who is allowed to touch the data'* (everyone) to *'who is responsible for the data'* (the card). The first version of the gate software in the scenario (negative balance, values typed in several places) is exactly what unrestricted global data produces. Section 7 of Part 2 runs both versions: after a careless `balance -= 9000` the procedural balance becomes **-2600 paise**, while the `SmartCard` version refuses the same attempt at compile time because `balance_` is private.

1.3 The four OOP principles and where they appear

Principle	Where it appears in this system	Why it helps
Encapsulation	`SmartCard::balance_` is private; only `deduct`, `recharge`, `canTravel` change it.	The Rs 50 rule and the no-negative rule cannot be bypassed.
Abstraction	The gate calls `card.deduct(fare)` without knowing how the balance is stored (paise, long long).	Storage can change (e.g. database) without touching gate code.
Inheritance	A possible `ConcessionCard : SmartCard` for students/seniors that reuses all card code. In the current system a `Category` enum is simpler, so inheritance is **optional**.	Shows reuse of the card rules; not forced where an enum is enough.
Polymorphism	A `FareRule` base class with `virtual long long apply(long long)` and derived `PeakRule`, `WeekendRule`, `ConcessionRule`; the gate loops over rules. (Used in the Day-4 discussion as a future option.)	New discount = new class, not a change to existing code.

1.4 Rules every correct journey must satisfy

Each rule is stated with the reason it exists; the tests in Part 2 check each one.

No.	Rule	Reason
R1	After paying, the balance must be **>= Rs 50.00 (5000 paise)**.	Given in the scenario; guarantees a return journey.
R2	The balance is never negative.	A negative balance appeared in version 1; R1 implies R2.

No.	Rule	Reason
R3	Base fare is Rs 10 / 20 / 30 by the **farthest** zone travelled.	Scenario definition.
R4	Peak = [08:00, 10:00) and [17:00, 20:00); peak adds 20 %.	Half-open intervals make 10:00 and 20:00 consistently off-peak (the old code was inconsistent).
R5	Students and senior citizens get a concession (**assumed 50 %**, applied after the surcharge).	The scenario gives no percentage; the value is a single constant and easy to change.
R6	All money is integer paise; percentages use integer arithmetic.	Removes 'off by a paisa' errors.
R7	A denied journey changes nothing (balance unchanged).	Atomic update; no partial deduction.
R8	Recharge must be positive.	A negative recharge would silently reduce the balance.
R9	Every fare value is defined once.	Revision in one place.

1.5 Alternative concept: purely procedural design - complexity analysis

Procedural solution. Global `long long balance;`, global `const` fare array and free functions `fare(zone, peak)`, `canTravel(f)`, `deduct(f)` operating on the global. This works for one card only; many cards need an array of balances and an index passed to every function.

Measure	Class-based (SmartCard, Journey, FareTable)	Procedural (globals + free functions)
Time per journey	O(1): one table lookup, a few integer operations	O(1): the same operations
Memory per card	O(1): 8 bytes (`balance_`) - measured `sizeof(SmartCard)` = 8	O(1): 8 bytes per balance in an array
Code changed when one fare rule changes	Edit one constant in `FareTable`; no function changes. Minimum balance: **1 line** (`MIN_BALANCE_PAISE`).	Every function that repeats the value must be found and edited; in the demo `minBalance` is a separate global that other code can also overwrite. Risk of missing one place.
Rule enforcement	Compiler enforces privacy	Only programmer discipline

Conclusion. Run time and memory are the same, O(1) in both. The object-oriented design is better because the cost that grows with the system is *maintenance*: a change touches one place instead of many. **Assumptions:** one card per passenger; peak decided by the tap-in time; fare decided by the farthest zone; concession = 50 %; time is local time at the gate.

Design v1 - UML class diagram (Namma Metro fare system)

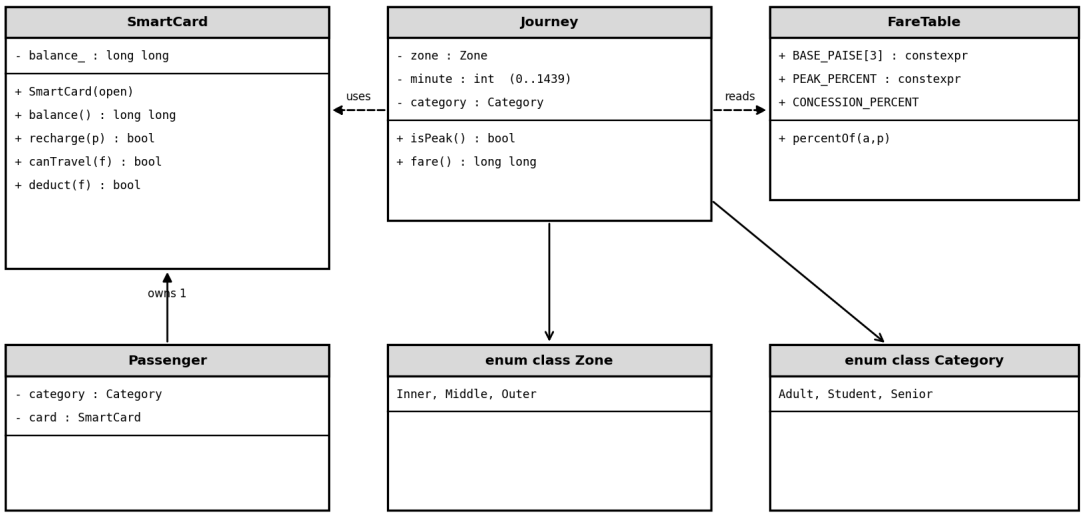


Figure 1 - UML class diagram of the original design (v1).

TASK 2 - Tokens, Keywords, Identifiers and Constants

3 Marks | K5 Evaluate

2.1 The five kinds of tokens

Every statement is broken by the compiler into tokens. Example line from the program: `constexpr long long MIN_BALANCE_PAISE = 5000;`

Token kind	Examples from the fare program	Notes
Keywords	`enum`, `class`, `constexpr`, `long`, `switch`, `case`, `return`, `bool`, `for`	Reserved by the language; cannot be used as names.
Identifiers	`fare`, `canTravel`, `BASE_PAISE`, `balance`, `SmartCard`	Names chosen by the programmer.
Constants (literals)	`1000`, `20`, `5000`, `'A'`, `"Denied"`, `true`	Fixed values written in the source.
Operators	`+`, `-`, `*`, `/`, `%`, `>=`, `&&`, `||`, `?:`, `+=`, `<<`	Perform computation / comparison.
Punctuators	`; { } ( ) [ ] , ::`	Separate and group; carry no value.

2.2 #define, const and constexpr

Feature	#define	const	constexpr
Handled by	Preprocessor (text replacement)	Compiler	Compiler
Type checking	None	Yes	Yes
Scope	From the definition to end of file; ignores namespaces and blocks	Normal C++ scope	Normal C++ scope
Compile-time value guaranteed	Yes (just text)	No: may be computed at run time	**Yes** - error if it cannot be computed at compile time
Usable for array size / `static_assert` / `case` labels	Yes	Only if initialised by a constant expression	Always
Debugging	Name vanishes before compilation	Name visible	Name visible

Why `constexpr` is preferred for the fare table: the fares are fixed before the program runs, the compiler checks their type (`long long`), the values can be used in `static_assert` (e.g. the table must have one entry per zone), and the names remain visible in the debugger. A macro such as `#define PEAK 20 + 5` would silently produce wrong results in `x * PEAK`, because the text is substituted without parentheses.

2.3 enum versus enum class

A plain `enum Zone { Inner, Middle, Outer };` puts `Inner` into the surrounding scope and converts to `int` silently, so `Zone z = ...; if (z == 7)` or `int fare = BASE[z + 1]` compiles. A second enum `enum Colour { Inner }` would clash. With `enum class Zone { Inner, Middle, Outer };` the names are written `Zone::Inner`, no implicit conversion to int exists, and two enums can reuse names. A `switch` over an `enum class` also lets `-Wswitch` warn when a case is missing (this is demonstrated in Part 2, Task 12). That is why zones and categories are `enum class`.

2.4 Naming rules and conventions

- **Rules (compiler):** letters, digits and underscore only; must not start with a digit; case-sensitive; no keywords; avoid names starting with underscore + capital or containing double underscore (reserved).
- **Conventions used here:** `UPPER_SNAKE_CASE` for constants (`MIN_BALANCE_PAISE`), `PascalCase` for types (`SmartCard`, `Zone`), `camelCase` for functions and variables (`canTravel`, `balance`), trailing underscore for private data members (`balance_`), and the unit in the name (`_PAISE`, `_PERCENT`) so that Rs and paise are never confused.

2.5 Evidence - the single set of constants

```
constexpr long long BASE_PAISE[]    = {1000, 2000, 3000};   // Rs 10, 20, 30  (one place only)
constexpr int  PEAK_SURCHARGE_PERCENT = 20;
constexpr int  CONCESSION_PERCENT     = 50;
constexpr long long MIN_BALANCE_PAISE = 5000;                 // Rs 50 must remain
constexpr int PEAK_AM_START = 8*60,  PEAK_AM_END = 10*60;     // [08:00, 10:00)
constexpr int PEAK_PM_START = 17*60, PEAK_PM_END = 20*60;     // [17:00, 20:00)
enum class Zone { Inner, Middle, Outer };
enum class Category { Adult, Student, Senior };
```

2.6 Alternative concept: #define macros - analysis

Macro solution: `#define BASE_INNER 1000`, `#define PEAK_PERCENT 20`, `#define INNER 0`, `#define MIDDLE 1`, `#define OUTER 2`. The program still runs, but the compiler no longer knows these are fares or zones.

Aspect	#define + int codes	constexpr + enum class
Type checking	None: `fare(7, true)` compiles; `INNER == 3` compiles	Zone values only; `fare(7, ...)` is a compile error

Aspect	#define + int codes	constexpr + enum class
Scope	Global text; clashes with any identifier of the same name in any header	Namespaced (`Zone::Inner`)
Memory	No storage (text substitution)	Usually no storage either (value folded into the instruction); an array like `BASE_PAISE[]` that is indexed at run time occupies 3 x 8 = 24 bytes
Running time	O(1)	O(1) - identical machine code after optimisation
Maintenance	Mistakes found late (or never)	Mistakes found at compile time

Conclusion: neither time nor memory differs in any practical way; the difference is type safety and scope, so `constexpr` and `enum class` are the justified choice.

TASK 3 - Choose Data Types and Check Type Compatibility

4 Marks | K5 Evaluate

3.1 Data-type table (with sizeof on this system)

Measured on g++ 13.3 (Ubuntu 24.04, 64-bit) with `task3_types.cpp`.

Value	Type chosen	sizeof	Range (from <climits>)	Reason
Fare (paise)	`long long`	8 B	-9.22e18 .. 9.22e18	Exact integer; same type as balances and totals, so no conversions.
Card balance (paise)	`long long`	8 B	-9.22e18 .. 9.22e18	Exact; can never hold a fraction of a paisa.
Time of day	`int` (minutes since midnight)	4 B	-2,147,483,648 .. 2,147,483,647 (needed: 0..1439)	One integer comparison per boundary; hour and minute are both kept.
Hour / minute input	`int`	4 B	same	Read with `cin`; validated 0-23, 0-59.
Zone number	`enum class Zone` (underlying int)	4 B	0..3	Closed set; type safe.
Peak / allowed flags	`bool`	1 B	true / false	Natural for yes/no.
Category code / letters	`char` or enum class	1 B / 4 B	-128 .. 127	Single characters only.
Journeys in a day	`long long` (counter)	8 B	up to 9.22e18	3,000,000 fits `int`, but a year (1.1e9) is near the `int` limit; `long long` removes the question.
Percentages	`int`	4 B	-2.1e9 .. 2.1e9	20, 50 - small whole numbers.
Display value only	`double` (never stored)	8 B	+-1.8e308	Used only to print Rs with 2 decimals, never for arithmetic.
`float`	not used	4 B	+-3.4e38	About 7 decimal digits: too coarse for money (shown in Task 10).
Loop items	`auto` / `const Trip&`	-	deduced	Avoids repeating long types; the type is still fixed at compile time.

3.2 Why money is kept as integer paise, not double rupees

A `double` stores numbers in binary. Rs 0.10 is 0.1000000000000000055... in binary, so it cannot be represented exactly; neither can Rs 0.01 or Rs 0.20. Integer paise (10, 1, 20) are represented exactly, and `+`, `-`, comparisons on integers are exact. The scenario says 'every fare and balance must be exact', and 'some fares are off by a paisa' is the classic symptom of `double` money.

3.3 Type-compatibility rules

- **Integral promotion:** `bool`, `char`, `short` become `int` in arithmetic (`'A' + 2` is an `int` 67).
- **Usual arithmetic conversions:** when types differ, the 'smaller' converts to the 'larger': `int` -> `long long` -> `float` -> `double`; if one operand is `unsigned int` and the other `int`, the `int` becomes `unsigned`.
- **Measured:** `3 + 2.5 + true` -> `3` becomes 3.0, `true` becomes 1 then 1.0, result `6.5` of type `double` (`typeid` name 'd').
- **bool conversion:** any non-zero value becomes `true` (1), zero becomes `false`.

3.4 Evidence - measured output

task3_types.cpp output

```
---- sizeof and range ----
bool        1 B
char        1 B  [-128, 127]
int         4 B  [-2147483648, 2147483647]
unsigned    4 B  [0, 4294967295]
long long   8 B  [-9223372036854775808, 9223372036854775807]
float       4 B  max 3.40282e+38
double      8 B  max 1.79769e+308

---- mixed-type expression ----
3 + 2.5 + true = 6.5 (type d, 'd' = double)

---- precision: add Rs 0.10 n times ----
         n | double rupees (10 dp) | exact (paise/100) | abs error
        10 |          1.0000000000 |      1.0000000000 | 1.110e-16
      1000 |        100.0000000000 |    100.0000000000 | 1.407e-12
    100000 |      10000.0000000188 |  10000.0000000000 | 1.885e-08
   1000000 |     100000.0000013329 | 100000.0000000000 | 1.333e-06
  10000000 |     999999.9998389754 | 1000000.0000000000 | 1.610e-04
0.10 stored as double = 0.10000000000000001
0.1+0.2 == 0.3 ? false

---- overflow: day's network collection in paise ----
3,000,000 x 2000 paise = 6000000000
INT_MAX               = 2147483647  -> int is NOT enough
ratio                 = 2.7939677251474784x INT_MAX
```

3.5 Precision and overflow problems with their resolution

Precision problem: adding Rs 0.10 repeatedly in `double` drifts away from the exact total as n grows (table above; plotted below). In integer paise the error is exactly 0 for every n. **Resolution:** store money as `long long` paise; convert to rupees only when printing.

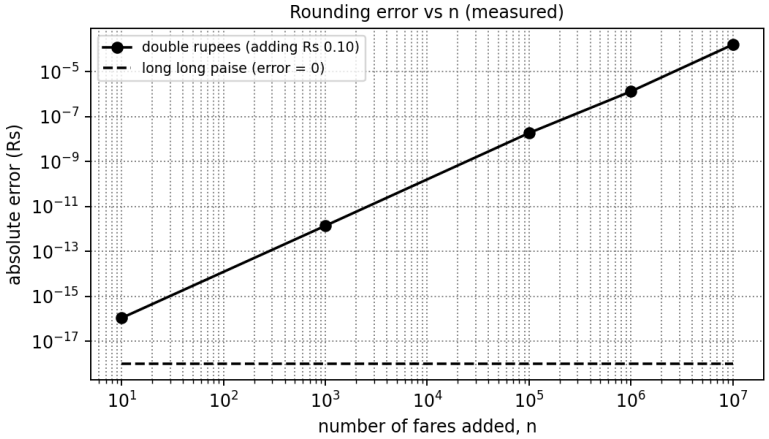


Figure 2 - absolute error of double-rupee accumulation (measured); paise error is 0.

Overflow problem: a day's network collection at an average of Rs 20 per journey is 3,000,000 x 2000 = **6,000,000,000 paise**, which is 2.79 times `INT_MAX` (2,147,483,647). A 32-bit `int` cannot hold it. **Resolution:** `long long` (max about 9.22e18, enough for roughly 2 billion such days). The measured int32 wrap-around is reported in Task 10.

3.6 Alternative concept: double rupees - analysis

Aspect	long long paise	double rupees
Time per addition	O(1)	O(1) (measured about 0.8 ns vs 1.5 ns per addition at -O2; same order)
Space per value	8 bytes	8 bytes
Exactness	Exact	Not exact for 0.10, 0.01, 0.20...
Rounding error after n additions	0	Grows with n: 1.1e-16 (n=10), 1.4e-12 (1e3), 1.9e-8 (1e5), 1.3e-6 (1e6), 1.6e-4 (1e7)
Equality test `0.1+0.2 == 0.3`	not needed	**false** in the measured run

Conclusion: time and space are the same, so the only difference is correctness. Integer paise is chosen.

TASK 4 - Control Implicit Conversions

4 Marks | K5 Evaluate

4.1 Where C++ converts without being told

- **Integer division then conversion:** `double c = a / b;` with `int a = 7, b = 2` first computes `7 / 2 = 3` (both operands are int), and only then converts 3 to `3.0`.
- **Signed/unsigned comparison:** `s < u` with `int s = -1`, `unsigned u = 1` converts `s` to unsigned: `-1` becomes `4294967295`, so the comparison is false. The compiler warns with `-Wsign-compare`.
- **Narrowing:** assigning a wider type to a narrower one (`int x = 3.7;` gives 3; `bool f = 5;` gives true). With brace initialisation `int x{3.7};` is a **compile error**.
- **char promotion:** `ch + 1` promotes `char` to `int`, so `std::cout` prints a number, not a letter.
- **static_cast:** needed when the conversion is intended: `static_cast<double>(a) / b` forces real division.

4.2 Predict the output (program given in the assignment)

```
#include <iostream>

int main()
{
int a = 7, b = 2;
double c = a / b;

double d = static_cast<double>(a) / b;
std::cout << c << ' ' << d << '\n';

char ch = 'A' + 2;
std::cout << ch << ' ' << ch + 1 << '\n';

bool flag = 5;
std::cout << flag << ' ' << std::boolalpha << flag << '\n';

unsigned int u = 1;
int s = -1;
std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

My predicted output (written by applying the rules above, before running):

```
3 3.5
C 68
1 true
s >= u
```

Actual output (compiled with `g++ -std=c++17 -Wall -Wextra`):

```
3 3.5
C 68
1 true
s >= u
```

Compiler warning produced: `warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]` (line 19).

4.3 Line-by-line explanation (prediction versus actual)

Output line	Conversion that took place	Explanation
`3 3.5`	Integer division, then int -> double	`a / b` is int / int = 3. `c` stores 3.0 and prints as 3 (cout drops the trailing .0). `d` casts `a` to double first, so 7.0 / 2.0 = 3.5. A common wrong guess is `3.5 3.5`.
`C 68`	int -> char; char -> int promotion	`'A' + 2` is int 67, stored in `char` as 'C'. `ch` prints as a character; `ch + 1` is an `int` (67 + 1 = 68) and prints as the number 68.
`1 true`	int -> bool	5 is non-zero, so `flag` is true (1). Without `boolalpha` it prints 1; after `std::boolalpha` it prints `true`. Note the manipulator affects the *rest of that statement* from the point it is written.
`s >= u`	signed -> unsigned	`-1` becomes 4294967295, which is greater than 1, so the condition is false and the `else` text is printed: surprising, because -1 < 1 mathematically.

Difference between prediction and actual: none; the surprises (`3` not `3.5`, `68` not `C`, `s >= u`) were all predicted from the conversion rules.

4.4 Corrected code (brace initialisation and static_cast)

```
// Task 4 - corrected version: no silent conversions
#include <iostream>

int main()
{
    int a{7}, b{2};
    double c{static_cast<double>(a) / b};          // intended real division
    double d{static_cast<double>(a) / static_cast<double>(b)};
    std::cout << c << ' ' << d << '\n';

    char ch{static_cast<char>('A' + 2)};
    std::cout << ch << ' ' << static_cast<int>(ch) + 1 << '\n';

    bool flag{true};                               // bool flag{5}; would NOT compile (narrowing)
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';

    unsigned int u{1};
    int s{-1};
    // compare as signed: convert the unsigned value deliberately
    std::cout << (s < static_cast<int>(u) ? "s < u" : "s >= u") << '\n';

    // int x{3.7};  -> error: narrowing conversion of '3.7' from 'double' to 'int'
    int x = static_cast<int>(3.7);                 // explicit: truncates to 3
    std::cout << "static_cast<int>(3.7) = " << x << '\n';
}
```

Output of the corrected program

```
3.5 3.5
C 68
1 true
s < u
static_cast<int>(3.7) = 3
```

Compiler errors proving brace initialisation blocks narrowing

```
../src/narrow_demo.cpp: In function 'int main()':
../src/narrow_demo.cpp:2:19: error: narrowing conversion of '3.7000000000000002e+0' from 'double' to 'int' [-Wnarrowing]
    2 | int main(){ int x{3.7}; bool f{5}; std::cout<<x<<f; }
      |                   ^~~
../src/narrow_demo.cpp:2:32: error: narrowing conversion of '5' from 'int' to 'bool' [-Wnarrowing]
    2 | int main(){ int x{3.7}; bool f{5}; std::cout<<x<<f; }
      |                                ^
```

4.5 Alternative concept: every conversion explicit - analysis

Solution: write every conversion as `static_cast<T>(...)` and every variable with `{}` initialisation. **Running time and memory do not change**: `static_cast<double>(a)` generates the same instruction (an int-to-double convert) as the implicit conversion, and no extra storage is used. **What changes is error detection:** narrowing (`int x{3.7}`, `bool f{5}`) is caught at **compile time** instead of silently losing data at run time; signed/unsigned mix-ups are visible in the source and silence the `-Wsign-compare` warning deliberately. The cost is longer code. For money the explicit style is recommended.

TASK 5 - Write Correct Expressions with the Right Precedence

4 Marks | K5 Evaluate

5.1 Fare expressions

```
constexpr bool isPeak(int hour, int minute = 0) {
    const int m = hour * 60 + minute;
    return (m >= 480 && m < 600) || (m >= 1020 && m < 1200);       // [08:00,10:00) or [17:00,20:00)
}
long long f = peak ? base + base * 20 / 100 : base;              // surcharge, integer arithmetic
if (category != Category::Adult) f -= f * 50 / 100;              // concession (compound assignment)
```

- **Precedence in `hour * 60 + minute`:** `*` is higher than `+`, so (hour * 60) + minute.
- **`base + base * 20 / 100`:** `*` and `/` have the same level and associate left to right: ((base * 20) / 100); then `+`. Multiplying first keeps the integer result exact (dividing first would lose the fraction).
- `peak ? a : b`**:** the conditional operator has lower precedence than `+` and comparison, and higher than assignment; it associates right to left.
- **`f -= f * 50 / 100`:** compound assignment has very low precedence, so the right side is fully evaluated first.
- `(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)` is correct. Because `&&` has higher precedence than `||`, the version without brackets, `hour >= 8 && hour < 10 || hour >= 17 && hour < 20`, groups the same way and is *equivalent*, but a reader must remember the rule; brackets show the two time windows at a

glance and protect against an edit that adds another condition.

5.2 x++ versus ++x, and i = i++

`x++` yields the **old** value and then increments; `++x` increments and yields the **new** value. In `z = x++ + ++y` with x = 5, y = 3: `x++` gives 5 (x becomes 6), `++y` gives 4 (y becomes 4), so z = 9. The statement `i = i++;` is a mistake: it modifies `i` twice in one expression (the increment and the assignment). Before C++17 the behaviour is undefined; in C++17 it is defined, but the result is that `i` keeps its **old** value, so the increment is lost. It should be written `i++;` or `i = i + 1;`.

5.3 Division and modulus with negative operands

Since C++11 integer division truncates toward zero and `%` takes the sign of the left operand, so that `(a / b) * b + a % b == a`: `10 / 3 = 3`, `10 % 3 = 1`, `-10 / 3 = -3`, `-10 % 3 = -1`. (Floor division would give -4 and 2; C++ does not do that.)

5.4 Predict the output

```
#include <iostream>

int main()
{
int x = 5, y = 3;
std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

int z = x++ + ++y;
std::cout << z << ' ' << x << ' ' << y << '\n';

std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

int k = 0;
for (int i = 0; i < 10; i += 3) k += i;
std::cout << k << '\n';
}
```

My predicted output (before running):

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Actual output:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line	Operators and order of evaluation	Result
`x + y * 2` and `(x + y) * 2`	* before +: 5 + (3*2) = 11. Parentheses first: (5+3)*2 = 16.	11 16
`z = x++ + ++y`	Post-increment and pre-increment (highest precedence) are done on their operands: x++ -> 5 (x=6), ++y -> 4 (y=4); then + -> 9; then = (lowest, right to left) stores 9 in z. Printed z, x, y.	9 6 4
`(x > y ? x : y)`	Parentheses force the `?:` expression first: 6 > 4 is true, so x = 6.	6
`x % 4 * 2`	% and * same level, left to right: (6 % 4) * 2 = 2 * 2 = 4.	4
`10 / 3`, `10 % 3`, `-10 / 3`, `-10 % 3`	Unary minus applies to 10 first, then / or %; truncation toward zero.	3 1 -3 -1
`for (i=0; i<10; i+=3) k += i;`	`i` takes 0, 3, 6, 9 (+= compound assignment); k = 0+3+6+9.	18

Difference between prediction and actual: none. The << operator has lower precedence than +, which is why `x + y * 2` needs no brackets inside the `cout` chain, but `x > y ? x : y` does.

5.5 Alternative concept: peak[24] lookup table - analysis

Solution: `constexpr bool PEAK[24] = {false x8, true, true, false x7, true, true, true, false x4};` and `bool peak = PEAK[hour];`. The program `task5_peak_table.cpp` compares it against the expression for all 24 hours: **0 mismatches**; the table occupies **24 bytes**.

Aspect	&& / || expression	peak[24] table
Time per check	O(1): up to 4 comparisons and 3 logical operators	O(1): one array index (fewer comparisons)
Extra space	none	24 bytes (1440 bytes for minute resolution)
Handles 10:00 vs 09:59	Yes (minutes since midnight)	Yes at hour resolution, because both windows start and end on the hour; a window such as 08:30 would need a 1440-entry (minute) table
Changing the peak window	Edit two constants	Edit several table entries by hand (easy to make an error)

Conclusion: the table is marginally faster but needs more edits and more space for finer times; the expression with named constants is retained.

TASK 6 - Design the Control Structures

4 Marks | K5 Evaluate

6.1 Choosing the structure

- **if-else** for conditions on ranges or combinations (peak or not, balance sufficient or not).
- **switch on the zone** when one value is compared with a small set of constant labels: clearer than a chain of `if (z == ...)` and the compiler can check that all enum values are covered (`-Wswitch`).
- **Conditional operator** `?:` for a short choice that produces a value: `peak ? base + base*20/100 : base`. Not used when either branch needs several statements.
- **Loops:** a *range-based for* (`for (const Trip& t : trips)`) is best for a fixed collection of journeys: no index, no off-by-one. A classic `for` is needed when the index matters (testing every hour 0..23). A `while` is best when the number of journeys is unknown and the loop runs until the input ends (`while (cin >> ...)`).
- **break / continue:** `continue` skips an invalid journey (unknown zone, bad time) and moves to the next one; `break` stops the day's processing when the input ends or the card is blocked. Each `case` in the `switch` ends with `break`, otherwise execution falls through to the next case.
- **No matching case:** if no `case` matches and there is no `default`, the `switch` does nothing and the variable keeps its initial value. In `fare()` that would leave `base = 0` and give a free journey, so zones are validated first (`zoneFromInt`) and the compiler warns when an enum value is missing.

6.2 Pseudocode (structured algorithmic format)

The pseudocode below uses a header (name, inputs, outputs), numbered steps, indentation for blocks and the keywords IF/ELSE, SWITCH, FOR EACH, RETURN. It follows the structure of the SIMATS Engineering pseudocode standard: algorithm header, declared inputs and outputs, one operation per line.

```
ALGORITHM  ComputeFare
INPUT      zone (Inner|Middle|Outer), peak (true|false)
OUTPUT     fare in paise (integer)
CONSTANT   BASE_PAISE = [1000, 2000, 3000];  PEAK_PERCENT = 20

 1  BEGIN
 2    SWITCH zone
 3      CASE Inner : base <- BASE_PAISE[0]
 4      CASE Middle: base <- BASE_PAISE[1]
 5      CASE Outer : base <- BASE_PAISE[2]
 6    END SWITCH
 7    IF peak THEN
 8      base <- base + (base * PEAK_PERCENT) / 100        // integer arithmetic
 9    END IF
10    RETURN base
11  END

ALGORITHM  ProcessDay
INPUT      list of trips, balance in paise
OUTPUT     fare printed and final balance
 1  FOR EACH trip IN trips
 2     f <- ComputeFare(trip.zone, trip.peak)
 3     IF balance - f >= MIN_BALANCE_PAISE THEN
 4         balance <- balance - f ;  PRINT fare, balance
 5     ELSE
 6         PRINT "Denied (low balance)", balance
 7     END IF
 8  END FOR
```

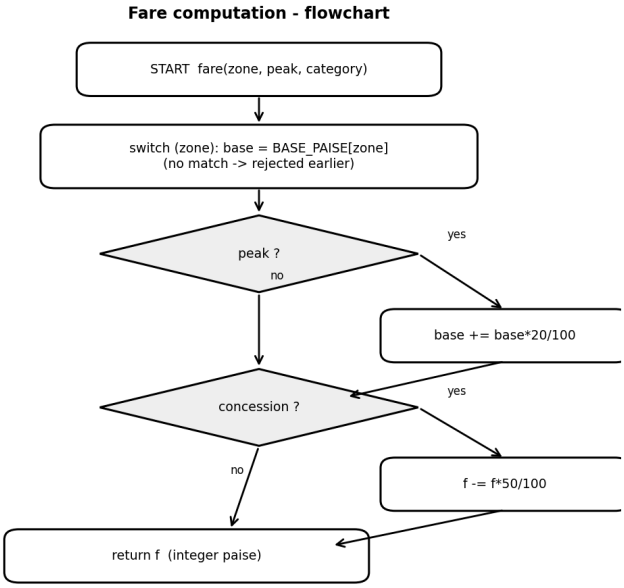


Figure 3 - flowchart of the fare computation.

6.3 Complete the Code - completed program and result

```
// Task 6 - Complete the Code (switch on zone, integer paise) + 10:00 / 20:00 boundary check
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
    long long base = 0;
    switch (z) {                                    // TODO 1
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
    }
    if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;   // integer arithmetic only
    return base;
}

bool canTravel(long long balance, long long f)
{
    return balance - f >= MIN_BALANCE_PAISE;       // TODO 2
}

constexpr bool isPeak(int hour, int minute)
{
    const int m = hour * 60 + minute;
    return (m >= 8 * 60 && m < 10 * 60) || (m >= 17 * 60 && m < 20 * 60);
}

int main()
{
    long long balance = 10000; // Rs 100.00 in paise
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                      << std::setfill('0') << f % 100 << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << " balance Rs " << balance / 100 << '\n';
    }

    std::cout << "\n--- boundary check: Inner zone, balance Rs 500 each time ---\n";
    struct T { int h, m; };
    for (T t : {T{7,59}, T{8,0}, T{9,59}, T{10,0}, T{16,59}, T{17,0}, T{19,59}, T{20,0}}) {
        bool pk = isPeak(t.h, t.m);
        std::cout << std::setfill('0') << std::setw(2) << t.h << ':' << std::setw(2) << t.m
                  << std::setfill(' ') << "  peak=" << pk << "  fare=" << fare(Zone::Inner, pk) << " paise\n";
    }
}
```

Output (first three lines match the expected output in the assignment)

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54

--- boundary check: Inner zone, balance Rs 500 each time ---
07:59  peak=0  fare=1000 paise
08:00  peak=1  fare=1200 paise
09:59  peak=1  fare=1200 paise
10:00  peak=0  fare=1000 paise
16:59  peak=0  fare=1000 paise
17:00  peak=1  fare=1200 paise
19:59  peak=1  fare=1200 paise
20:00  peak=0  fare=1000 paise
```

Boundary rule. The peak test uses half-open intervals, `m >= 480 && m < 600` and `m >= 1020 && m < 1200`. Therefore 10:00 is **off-peak** and 20:00 is **off-peak**, consistently (the original software accepted 10 AM as peak and 8 PM as off-peak inconsistently). The program output above shows 07:59 off-peak, 08:00 peak, 09:59 peak, 10:00 off-peak, 17:00 peak, 19:59 peak, 20:00 off-peak.

6.4 Alternative concept: array lookup instead of switch - analysis

Solution: `constexpr long long FARE[] = {1000, 2000, 3000};` and `base = FARE[static_cast<int>(z)];`.

Aspect	switch on zone	array indexed by zone
Time per lookup	O(1) (compiler may build a jump table)	O(1): one indexed load
Space	code only (a few bytes of jump table)	3 x 8 = 24 bytes
Adding a fourth zone	Add a `case`; `-Wswitch` **warns** if forgotten (shown in Task 12); forgotten case gives fare 0	Add one element; if forgotten, the index is out of bounds (undefined behaviour) unless a `static_assert` ties the size to the zone count
Invalid zone value	falls out of the switch (can be given `default`)	undefined behaviour
Readability	explicit label per zone	compact; data separated from logic

Conclusion: the switch is safest for three zones with fixed rules; for four or more zones (Task 12) the table indexed by zone is better, provided a `static_assert` guards its size.